Project 2: Modelling daily cycling demand in Edinburgh

Group members: Alice Miller, Saba Sitter, Jake Saunders

Word count: [XXX words]

1 Introduction

The "cycle daily" dataset consists of approximately 2,180 daily observations ranging from 2020 to 2025, sourced from Edinburgh's network of automatic cycling counters, along with local weather and temporal data. This data comes from sensors at specific locations across the city, so it's possible not every cyclist is accounted for. However, they are a good indicator of how cycling demand varies across different days and weather patterns. Edinburgh relies heavily on public transport for its residents and tourists to navigate the city. For future funding allocation and planning, it is important to know how many cyclists are on the road in order to evaluate whether the infrastructure (cycle lanes, crossings etc) and investment in schemes (rentable Voi bikes) need to adapt to meet demand. The modelling goal is to develop a sequence of increasingly complex linear regression models to isolate how temporal and weather variables, such as the day of the week and the temperature, influence cycling demand. Ultimately, the goal is to extrapolate cycling demand to the future so that Edinburgh City Council and Transport Scotland can optimise infrastructure investment over the next decade to align with their active travel priorities.

2 Exploratory data analysis

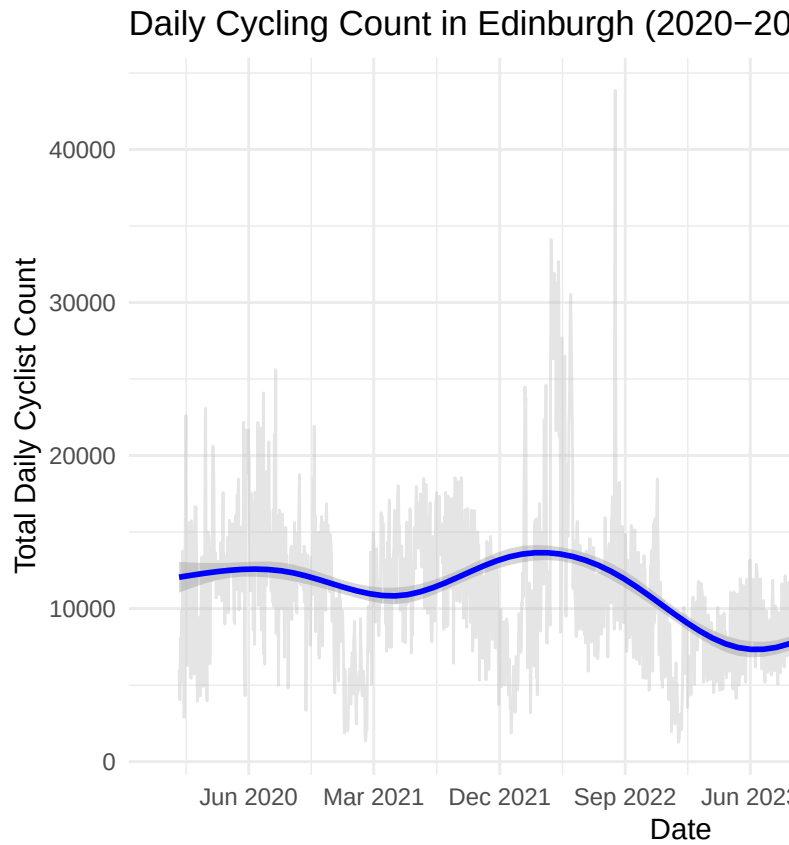
To begin investigating the impact of weather on the number of daily cyclists, we begun with some exploratory data analysis. A table displaying the summary statistics for the weather data and number of daily cyclists is as follows:

#summary statistics table for count, temp_mean, temp_min, temp_max

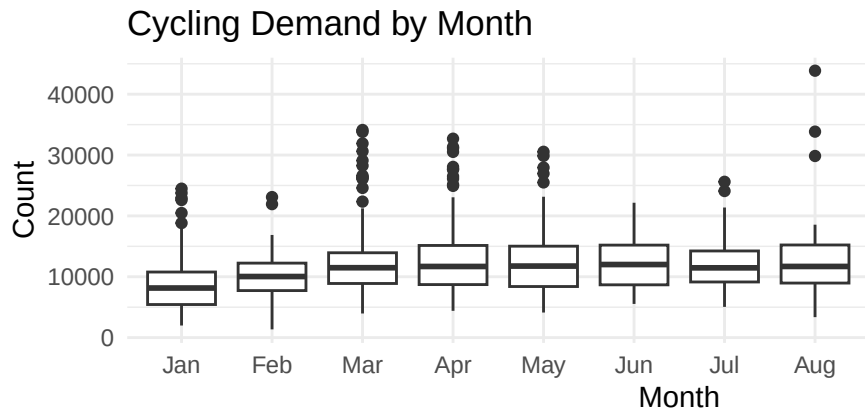
```
## # A tibble: 4 x 5
##   Variable      Mean      SD     Min     Max
##   <chr>        <dbl>   <dbl>   <dbl>   <dbl>
## 1 count     10815.   4729.   1264    43864
## 2 temp_mean     9.69    4.79   -8.08    23.0
## 3 temp_min      5.92    4.98  -14       18
## 4 temp_max     13.2     5.31   -3       31
```

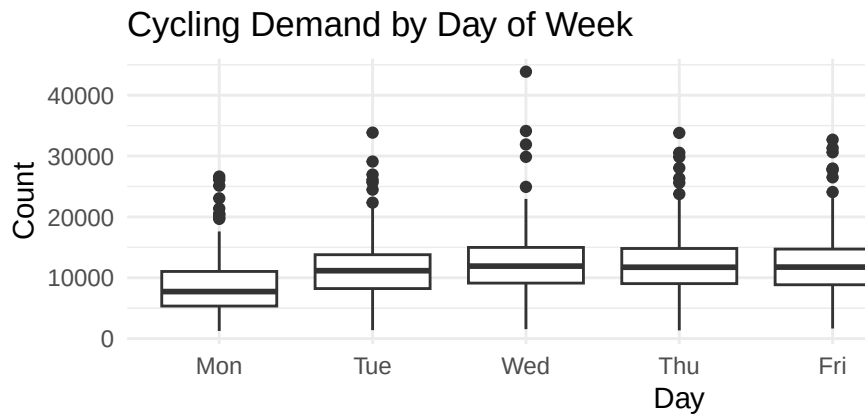
We can see that the number of daily cyclists varies from $1,264$ to $43,864$, an increase of $\approx 3470\%$, and the variation is very high. Intuitively, one would think that a warmer average weather would lead to more cyclists.

We then plotted a time series of the daily cyclist count from 2020 to 2025.



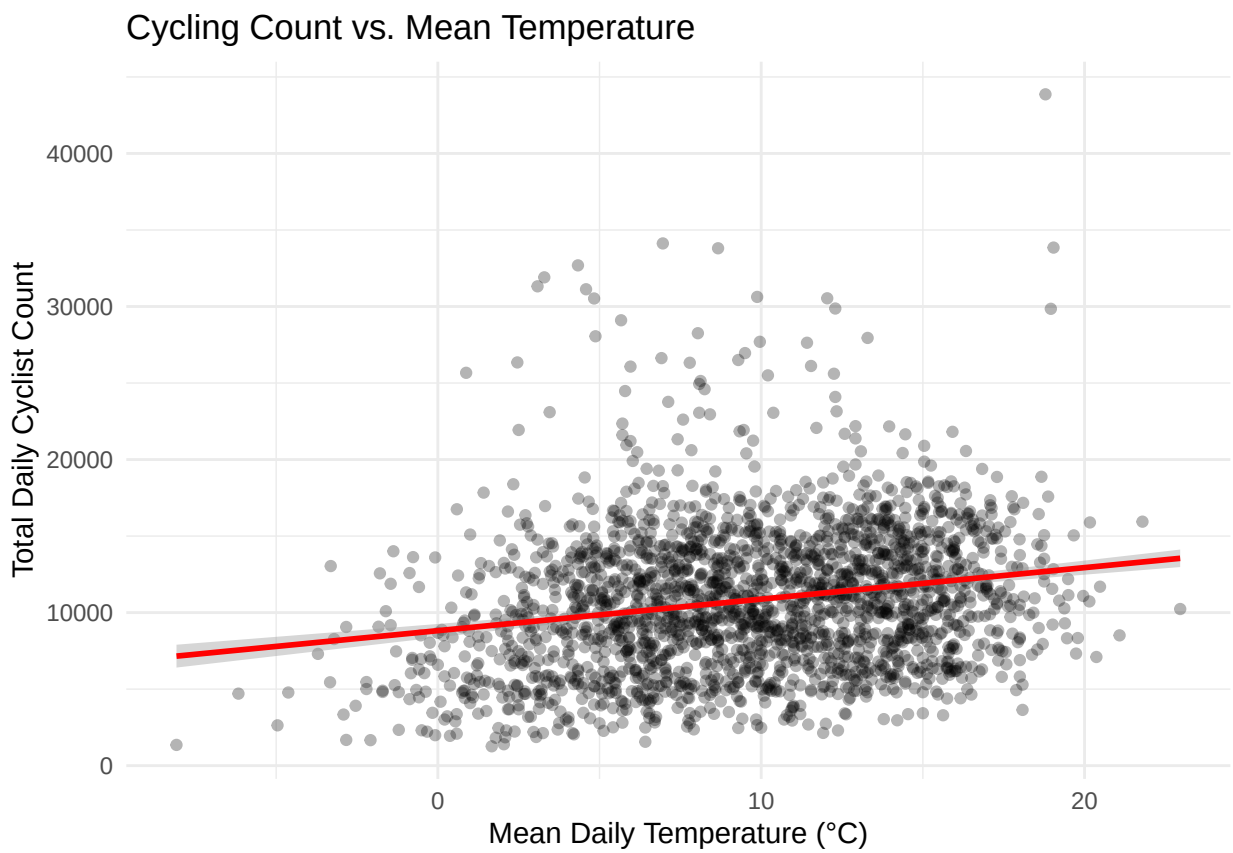
#Time series of daily counts 2020-2025 with a smoother.
We can see from the time series that there are yearly fluctuations, with peaks and troughs occuring yearly. Overall, the cyclist count seems to be decreasing. There is a visible peak in 2022, which we investigate. We then created boxplots of the daily cyclist count by month and by the day of week.





#Boxplots of count by month and by day of week.
We can see from these plots that the winter months have slightly lower cyclist counts, with the other months remaining relatively constant. This indicates that the month does not have a tremendous impact on the number of daily cyclists outside of January and December. From the second boxplot, there appear to be less cyclists on Mondays and Sundays, with the other days again appearing relatively constant.

#Scatter plot of count vs temp_mean.



#At least one further plot of your choice highlighting a notable feature.

#Every figure must have a caption and be explicitly discussed in the text.

Table 1, shows the summary statistics for the base dataset. It gives the mean, standard deviation (SD), the minimum value and the maximum value for count (number of cyclists per day), the temperature mean, temperature minimum and temperature maximum.

Table 1: Summary statistics for daily cyclist counts and temperature variables

Variable	Mean	SD	Min	Max
count	10815.4739	4728.8841	1264.0000	43864.0000
temp_mean	9.6914	4.7908	-8.0833	22.9583
temp_min	5.9216	4.9755	-14.0000	18.0000
temp_max	13.1505	5.3075	-3.0000	31.0000

2.1 Time Series Analysis

Figure 1 shows the daily cycling counts from 2020 to 2025.

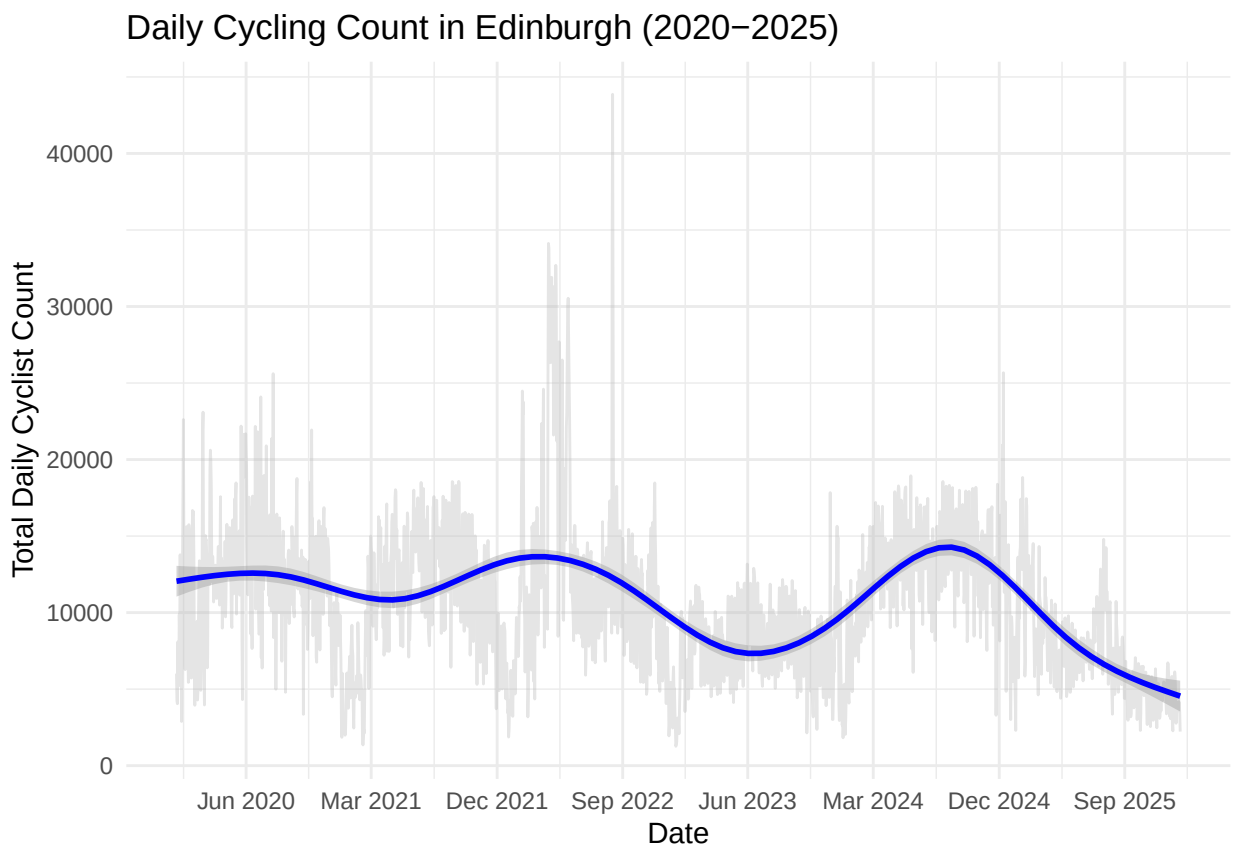


Figure 1: Daily cycling counts in Edinburgh with a smoother.

2.2 Seasonal and Weekly Patterns

Figures 2 and 3 illustrate how cycling demand changes across months and days of the week.

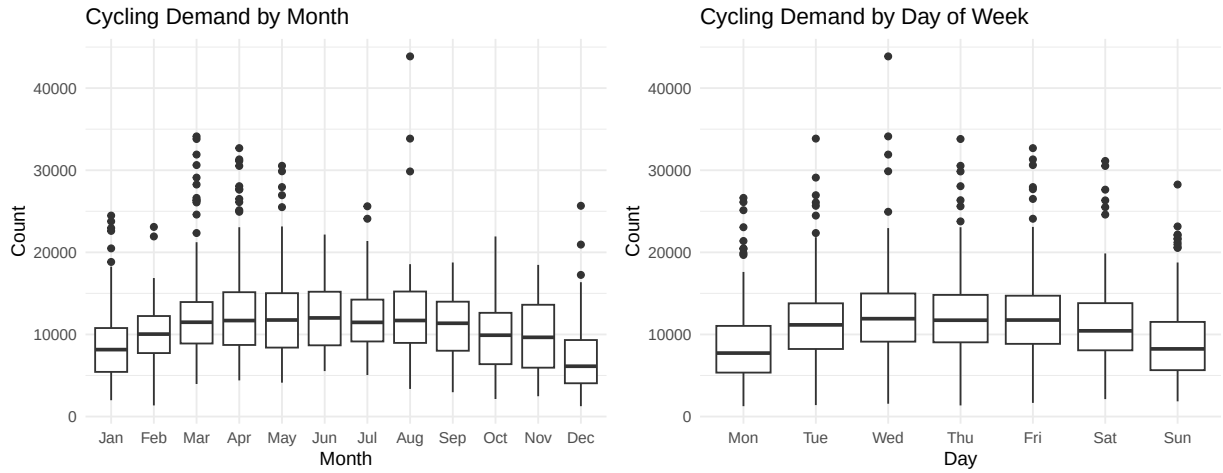


Figure 2: Distribution of daily cyclist counts by month (left) and day of week (right).

Figure 4 shows the relationship between temperature and daily cyclist counts.

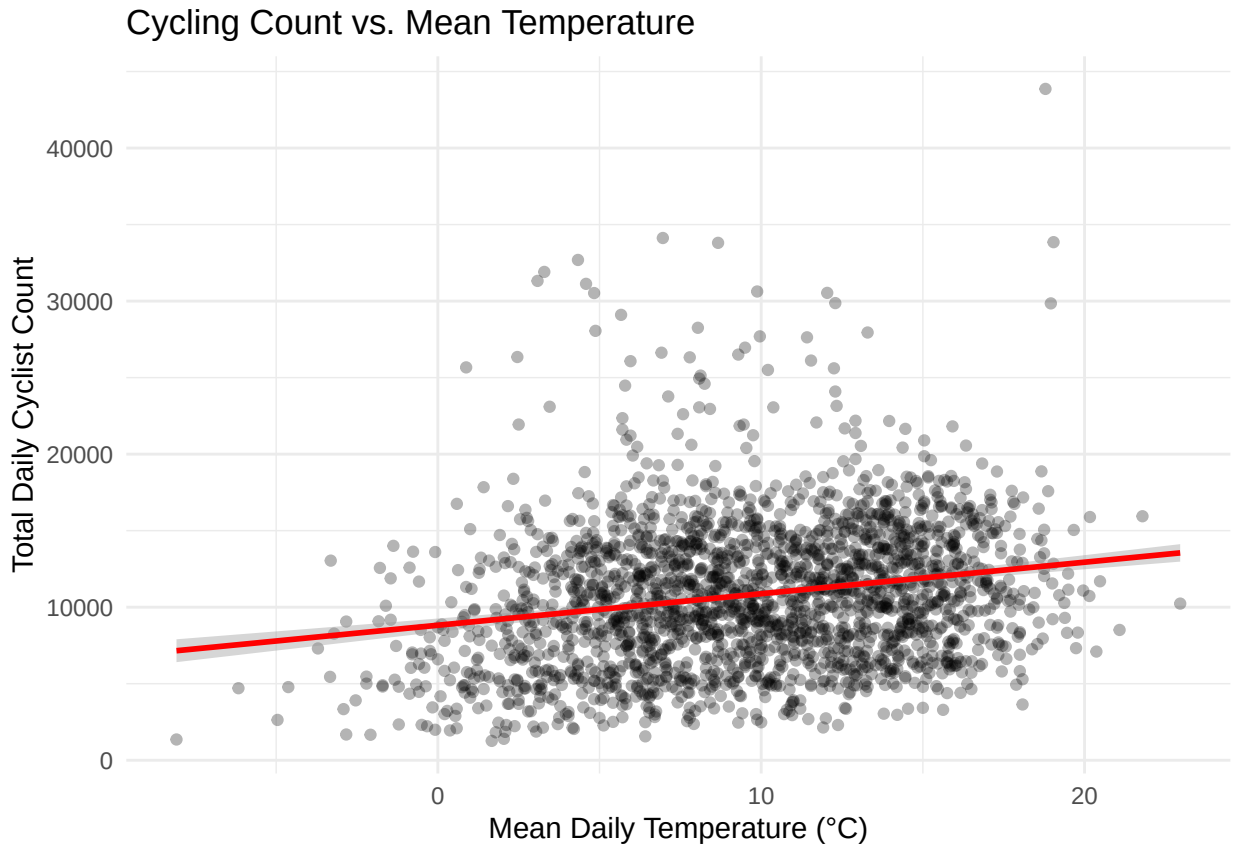


Figure 3: Scatter plot of daily cyclist counts against mean temperature.

3 Baseline model M0

4 Proposed models M1–M3

4.1 Model M1: Trend and factor

4.2 Model M2: Nonlinear temperature

4.3 Model M3: [your extension title]

5 Evaluation

5.1 Statistical Performance

5.2 Practical Performance

6 Discussion and limitations

7 Appendix: Code

All analysis code is in `code.R` and is reproduced below.

```
# ============================================================
# Project 2: Modelling daily cycling demand in Edinburgh
# Authors: Alice Miller, Saba Sitter, Jake Saunders
# ============================================================

#
# Place the code needed in the Report_project02.Rmd, including documentation.
#
# 0. Packages
library(ggplot2); library(dplyr); library(tidyr)
library(lubridate); library(knitr); library(kableExtra)
library(patchwork); library(broom)

# 1. Load data
load('cycle_daily_df.Rdata')

# 2.1 Data Wrangling

cycle_daily_df <- cycle_daily_df %>%
  mutate(
    # Task 1: month as ordered factor for plots/inference
    month = factor(month,
                   levels = 1:12,
                   labels = month.abb,
                   ordered = TRUE),

    # Task 2: dow with explicit levels
```

```r
    dow = factor(dow,
                 levels = c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"),
                 ordered = TRUE),

    # Task 3: trend as integer days since 2020-01-01
    trend = as.numeric(date - date[1], units = "days")
    )

# 2.2 exploratory plots

# Summary table for count, temp_mean, temp_min, temp_max
summary_stats <- cycle_daily_df %>%
  summarise(across(c(count, temp_mean, temp_min, temp_max),
                   list(Mean = ~mean(.x),
                        SD = ~sd(.x),
                        Min = ~min(.x),
                        Max = ~max(.x)),
                   .names = "{.col}.{.fn}")) %>% # use dots in variable names instead of underscore
  # Reshape for a vertical table
  pivot_longer(everything(),
               names_to = c("Variable", "Statistic"),
               names_sep = "\\.") %>% # Split at the dot instead of the underscore
  pivot_wider(names_from = Statistic, values_from = value)


# Time series of daily counts 2020-2025 with a smoother
plot_timeseries <- ggplot(cycle_daily_df, aes(x = date, y = count)) +
  geom_line(alpha = 0.4, color = "gray") +
  geom_smooth(method = "gam", color = "blue", se = TRUE) + # The smoother line
  scale_x_date(date_breaks = "9 months", date_labels = "%b %Y") +
  labs(
    title = "Daily Cycling Count in Edinburgh (2020-2025)",
    x = "Date",
    y = "Total Daily Cyclist Count"
  ) +
  theme_minimal()

# Boxplots of count by month and by day of week.

# Boxplot by Month
plot_month <- ggplot(cycle_daily_df, aes(x = month, y = count)) + # Use fill = month to add colour
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Cycling Demand by Month", x = "Month", y = "Count") +
  theme_minimal()

# Boxplot by Day of Week
plot_dow <- ggplot(cycle_daily_df, aes(x = dow, y = count)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Cycling Demand by Day of Week", x = "Day", y = "Count") +
  theme_minimal()

# Scatter plot of count vs mean temperature
plot_temp_scatter <- ggplot(cycle_daily_df, aes(x = temp_mean, y = count)) +
```

```r
  geom_point(alpha = 0.3) + # alpha changes the opaqueness of the points
  geom_smooth(method = "lm", color = "red", se = TRUE) +
  labs(
    title = "Cycling Count vs. Mean Temperature",
    x = "Mean Daily Temperature (°C)",
    y = "Total Daily Cyclist Count"
  ) +
  theme_minimal()

#plot(cycle_daily_df$date,cycle_daily_df$count)
#boxplot(count ~ month , data = cycle_daily_df)
#boxplot(count ~ dow , data = cycle_daily_df)



# 3. Model Fitting

# set the seed, defined test and training data,
n = 2192
set.seed(1234) # for reproducibility
idx <- sample(1:n, size = round(0.7*n))
train <- cycle_daily_df[idx, ]
test  <- cycle_daily_df[-idx, ]

# Note: Use factor(month) in formulas for M1-M3
m0 <- lm(count ~ temp_mean + as.numeric(weekend) + as.numeric(month), data = train)
## m0 + factor(month) unsure what double parameters - choose one to avoid collinearity
m1 <- lm(count ~ temp_mean + as.numeric(weekend) + trend +
           factor(month) + factor(dow), data = train)
m2 <- lm(count ~ temp_mean + weekend + month + trend + factor(month) + factor(dow) + I(temp_mean^2), da
#m3 <- lm()



#diagnostic plots for first 3 models
#
#  Residuals vs fitted and normal QQ diagnostic plots, side by side.
#  Brief assessment: what does M0 get right and wrong?
#  What motivates M1?

#m0

par(mfrow = c(2, 2))
plot(m0)

#m1
plot(m1)

#m2
plot(m2)
```

```r
# 4. Cross-Validation Functions

calc_scores <- function(y, mu, sigma, alpha = 0.05) {
  # y     : vector of observed values
  # mu    : vector of predictive means (from predict(fit, newdata=test)$fit)
  # sigma : vector of predictive SDs. Combine residual error and mean uncertainty:
  #         sigma = sqrt(summary(fit)$sigma^2 +
  # predict(fit, newdata=test, se.fit=TRUE)$se.fit^2)
  # Returns a named list with RMSE, MAE, DS, IS

  # Implement RMSE, MAE, DS, and IS here

  #RMSE
  RMSE <- sqrt(mean((y - mu)^2))

  #MAE
  MAE <- mean(abs(y - mu))

  #DS
  DS <- mean(log(sigma^2) + ((y - mu)^2 / sigma^2))

  #IS
  lower <- mu - (qnorm(1-alpha/2) * sigma)
  upper <-mu + (qnorm(1-alpha/2) * sigma)
  IS <-  mean((upper - lower) +
    (2/alpha)* pmax(0,(lower - y)) +
    (2/alpha)*pmax(0,(y - upper)))

  # make a table with the results
  return (list(RMSE = RMSE, MAE = MAE, DS = DS, IS = IS))
}

# calc scores for m0
y = test$count
mu0 = predict(m0, newdata=test)
pred_obj <- predict(m0, newdata = test, se.fit = TRUE)
sigma0 = sqrt(summary(m0)$sigma^2 + pred_obj$se.fit^2)

calc_scores(y = y, mu = mu0, sigma = sigma0, alpha = 0.05)

y = test$count
mu0 = predict(m1, newdata=test)
pred_obj <- predict(m1, newdata = test, se.fit = TRUE)
sigma0 = sqrt(summary(m1)$sigma^2 + pred_obj$se.fit^2)

calc_scores(y = y, mu = mu0, sigma = sigma0, alpha = 0.05)
```

```
# 5. Leave-One-Year-Out CV Loop
# 6. CV by Month
# ...
```